# GPU Optimization & Resource Report

Face Detection + Blurring Pipeline | Project: AI Research / Real-time Video Privacy

## 1. Device Settings

| Setting | Value |
|---|---|
| Host CPU | Intel Core i7-14700K (28 logical cores, 31 GB RAM) |
| GPU | NVIDIA GeForce RTX 5080 (16 GB VRAM) |
| Driver / CUDA | 595.99.02 \| CUDA 12.9 |
| Runtime | DeepStream + TensorRT (FP16) |
| Model | RetinaFace-R50 \| input 1280 × 736 \| conf 0.15 / NMS 0.4 |
| Source | Live RTSP stream |
| Frames processed | 5 058 (30 warm-up frames excluded from timings) |
| Run duration | 169.1 s |

## 2. Performance Summary

| Metric | Value |
|---|---|
| Avg Pipeline FPS (steady-state) | 30.0 |
| Whole-run FPS | 30.1 |
| Estimated Max FPS | 518.2 |
| Headroom | 17.3× |
| Frames Processed / Lost | 5 058 / 0 |
| Keeping up with stream | YES |
| End-to-end latency (mean / p95 / max) | 6.4 ms / 7.9 ms / 19.9 ms |

*Pipeline is source-limited at ~30 FPS. Estimated max capacity ~518 FPS (calculated from the stage timings; the slowest stage is TensorRT inference).*

## 3. Important Modules – GPU vs CPU

*Only the modules that matter for understanding where the work happens are listed.*

| Module | Device |
|---|---|
| Video Decode (NVDEC) | GPU |
| Face Detection (TensorRT / RetinaFace-R50) | GPU |
| Post-process (ROI / NMS / box decode kernels) | GPU |
| Face Blur (custom CUDA kernels) | GPU |
| Video Encode (NVENC) | GPU |
| JPEG encode (nvJPEG) | GPU |
| Stream Mux / Queues (DeepStream) | GPU |
| Live preview (nvvideoconvert + nvdsosd + EGL window) | GPU |
| Detection & audit log writing | CPU |
| Probe callbacks & Python glue | CPU |

*Almost the entire compute path runs on GPU, including the live preview. CPU handles lightweight bookkeeping, logging and launching GPU jobs.*

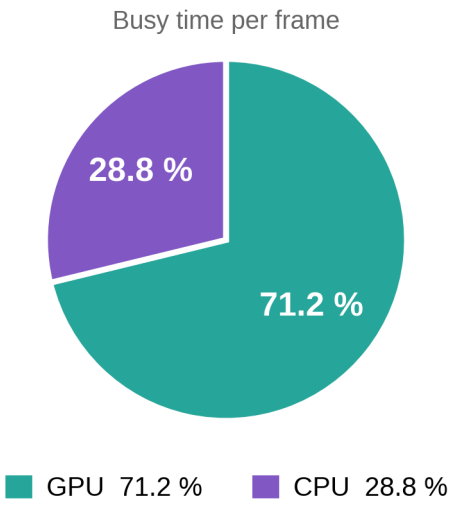
## 4. Resource Usage – Before / During / After Pipeline

*Samples taken at start (init + warm-up), steady state (main run), and at finish.*

| Metric | At Start | During (Steady) | At Finish |
|---|---|---|---|
| Process CPU (cores) | 0.79 | 0.16 | 0.28 |
| System CPU (all cores %) | 5 % | 2 % | 2 % |
| GPU utilisation avg | 1 % | 6 % | 6 % |
| GPU utilisation peak | — | 8 % | — |
| GPU memory used | 3 936 MB | 4 947 MB | 4 098 MB |
| GPU power | 56 W | 75 W | 75 W |
| GPU temperature | 37 °C | 41 °C | 42 °C |
| Process RAM (RSS) | 883 MB | 1 483 MB | 1 199 MB |

## 5. Workload Split (GPU vs CPU)

Measured busy time per frame in steady state = 5.50 ms (GPU 3.92 ms + CPU 1.58 ms). Host waiting for GPU is only 0.06 ms and is counted separately.




**GPU 71.2 % of the busy work · CPU 28.8 % of the busy work.**

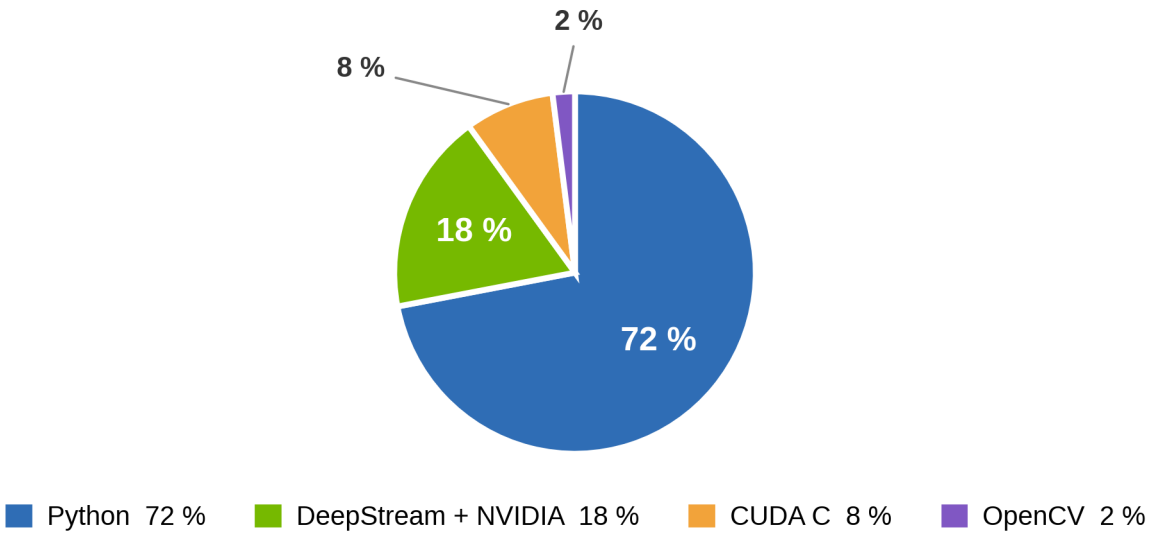
Of the GPU share, TensorRT inference + NVENC account for the majority; the remaining GPU time is custom CUDA kernels (blur, NMS, ROI), nvJPEG and the GPU live-preview branch.

## 6. Top Cost Items (ms per frame)

| Stage | Device | ms / frame |
|---|---|---|
| TensorRT inference (nvinfer) | GPU | 1.93 |
| Kernel launch / enqueue overhead | CPU | 1.11 |
| NVENC video encode | GPU | 0.80 |
| ROI build + compaction (CuPy) | GPU | 0.55 |
| Top-K sort + gather (CuPy argsort) | GPU | 0.23 |

## 7. Code Composition (Estimate)

*Estimated share of the whole code base by role (not a line count).*




| Category | Share | What it covers |
|---|---|---|
| **Python** | **72 %** | Almost everything: classes, pipeline building, probes, logging, argument parsing, control logic, JPEG saver orchestration, report, etc. |
| **DeepStream + NVIDIA** | **18 %** | All the DeepStream elements (nvurisrcbin, nvstreammux, nvinfer, nvvideoconvert, NVENC, nvdsosd, nveglglessink…), pyds, TensorRT engine, NVDEC/NVENC, CuPy usage |
| **CUDA C** | **8 %** | The real kernels inside CUDA_SRC (blur_h, blur_v, rf_decode, nms_mask, nms_reduce) |
| **OpenCV** | **2 %** | Only the small CPU fallback for JPEG saving + the one-time colour check |

## Summary

RetinaFace-R50 pipeline on live RTSP source ran at **30.0 FPS** (source-limited). 0 frames lost. Pipeline headroom **17.3×** (estimated max ≈ 518 FPS). GPU average utilisation only 6 %; process CPU ≈ 0.16 cores. Busy-time workload split: **GPU 71.2 % / CPU 28.8 %**. Almost all heavy lifting (decode, detection, blur, encode, JPEG saving and the live preview) stays on the GPU; CPU is used mainly for launching GPU jobs, light Python orchestration and logging.